MIT 8301 Virtual Laboratory

German Credit Risk Classification

Student	Basil Oforbuike Emeokoro
Matric Number	2025/A/MIT/0111
Student ID	301806653
Email	b.emeokoro6653@miva.edu.ng
Programme	Master of Information Technology
Specialisation	Artificial Intelligence & Machine Learning
Course	MIT 8301 - Machine Learning and AI Fundamentals
Academic Session	2026/2027

Cover Page and Student Details

German Credit Risk Classification

Student: Basil Oforbuike Emeokoro

Matric Number: 2025/A/MIT/0111

Student ID: 301806653

Email: b.emeokoro6653@miva.edu.ng

Programme: Master of Information Technology

Specialisation: Artificial Intelligence & Machine Learning

Course: MIT 8301 - Machine Learning and AI Fundamentals

Academic Session: 2026/2027

Executive Summary

This assessment develops a reproducible classifier for bad credit risk. The supplied file contained 1,000 applicants and nine predictors but omitted the label. An authentic target-inclusive edition was reconciled with a 100% exact row-level predictor match and zero mismatched cells before labels were appended. The verified target contains 700 good and 300 bad applicants. Four models were trained and evaluated with leakage-safe preprocessing. Tuned Logistic Regression was selected under a predeclared performance-and-governance rule.

Problem Statement and Objectives

The objective is to identify risky applicants (credit_risk=1) while recognising the asymmetric institutional and fairness costs of false negatives and false positives. The work covers EDA, missing values, outliers, encoding, scaling, scratch Logistic Regression, tuned model comparison, transparent feature interpretation, and responsible-use limitations.

Dataset Description, Provenance, and Target Restoration

The supplied export had 1,000 rows, an index column, and nine predictors. The target source was the Hugging Face mirror of Kaggle's German Credit Risk - With Target edition. After removing the export index and normalising only whitespace/case, every predictor cell matched. No fuzzy or positional merge was used. good was encoded 0 and bad 1. SHA-256 checksums and per-feature match percentages are recorded in the provenance report.

Exploratory Data Analysis

There are no duplicated complete rows. Age has 53 unique values, credit amount 921, and duration 33. The target is moderately imbalanced (70% good, 30% bad). Histograms, robust-scaled boxplots, categorical distributions, a missingness chart, a correlation heatmap, target comparisons, and observed risk by purpose appear in the figures appendix. These are descriptive associations, not causal findings.

Missing Values and Outliers

Savings status has 183 missing values; checking status has 394. A dedicated Unknown_or_No_Account category preserves potentially informative absence and is learned inside training folds. IQR review found {'Age': 23, 'Credit amount': 72, 'Duration': 70}. Values are plausible, so they are not deleted. Training-fitted 1.5-IQR clipping limits leverage, followed by robust scaling. This avoids data leakage.

Encoding, Scaling, and Split

The data were split into 800 training and 200 untouched test rows with stratification and random state 42. Age, amount, and duration use median imputation, IQR clipping, and RobustScaler. Nominal fields, including Job, use constant imputation and one-hot encoding with unknown-category tolerance. All transformations are fitted only on training data or within cross-validation folds.

Logistic Regression from Scratch

The implementation provides a stable clipped sigmoid, explicit bias, binary cross-entropy, vectorised gradients, L2 regularisation, convergence tolerance, loss history, probability estimates, and input/fitted-state validation. It converged=True in 4233 iterations with final loss 0.50469. Its test performance is compared directly with scikit-learn.

Model Training and Hyperparameter Tuning

Training used stratified five-fold GridSearchCV and ROC-AUC as the primary score. The test set was never used for tuning. Tuning controls regularisation, margin/kernel complexity, class weighting, and Naive Bayes variance smoothing, improving generalisation without test leakage.

• **Tuned Logistic Regression:** CV ROC-AUC 0.756; {'model__C': 0.1, 'model__class_weight': None, 'model__penalty': 'l2', 'model__solver': 'liblinear'}

• **Tuned SVM:** CV ROC-AUC 0.765; {'model__C': 1, 'model__class_weight': 'balanced', 'model__gamma': 0.1, 'model__kernel': 'rbf'}

• **Tuned Gaussian Naive Bayes:** CV ROC-AUC 0.666; {'model__var_smoothing': 1e-12}

Test Evaluation and Comparison

Model	Accuracy	Precision	Recall	F1	ROC-AUC

|---|---:|---:|---:|---:|---:|

Tuned SVM	0.710	0.511	0.750	0.608	0.786
Tuned Logistic Regression	0.730	0.594	0.317	0.413	0.766
Logistic Regression from Scratch	0.720	0.548	0.383	0.451	0.756
Tuned Gaussian Naive Bayes	0.675	0.472	0.700	0.564	0.702

A false negative classifies a risky applicant as good and may expose the institution to loss. A false positive classifies a good applicant as risky and may cause unfair rejection or harsher terms. Accuracy alone is therefore insufficient; recall, F1, ROC-AUC, thresholds, interpretability, and operating costs matter.

Best-Model Justification

The predeclared rule prefers tuned Logistic Regression when its ROC-AUC is within 0.03 of the highest model, otherwise the highest-AUC model is selected. The result is Tuned Logistic Regression. This accepts a small discrimination trade-off for transparent coefficients, simpler deployment, auditability, and regulatory suitability. Operational threshold tuning could improve risky-class recall and must be set from validated costs, not the test set.

Feature Importance and Interpretation

The largest absolute transformed Logistic Regression coefficients are:

• cat__Checking account_Unknown_or_No_Account: -0.879

• num__Duration: +0.514

• cat__Checking account_little: +0.471

• cat__Saving accounts_Unknown_or_No_Account: -0.392

• cat__Purpose_radio/tv: -0.360

• cat__Housing_own: -0.343

• cat__Sex_male: -0.330

• cat__Saving accounts_little: +0.313

• cat__Purpose_education: +0.307

• cat__Saving accounts_rich: -0.283

Positive values increase estimated bad-credit log-odds and negative values decrease them, conditional on the encoding and scaling. Coefficients express association, not causation. One-hot terms are interpreted relative to the omitted all-zero representation.

Ethics, Fairness, and Responsible Use

Sex is sensitive and age can create protected-class concerns. Historical labels may encode past discrimination and other fields may serve as proxies. Supplementary metrics by sex are reported but small test subgroups cannot establish fairness. The model requires human review, reasons for adverse decisions, appeal routes, privacy controls, drift monitoring, periodic bias audits, and independent validation. Prediction is not causal judgement.

Limitations

The dataset is small, historical, geographically specific, and omits important affordability and contemporary lending variables. Labels have no event-time detail; calibration and temporal stability are not established. The simplified nine-feature representation loses information from the original 20-feature UCI data. Reported test estimates have sampling uncertainty.

Conclusion

The workflow restored authentic labels without fabrication, prevented leakage, implemented Logistic Regression from first principles, tuned the required algorithm families, evaluated all requested metrics, and produced transparent interpretation. The result is suitable for assessment and portfolio demonstration, not production lending.

Reproducibility

Python 3.13.6; pandas 2.3.0; NumPy 2.3.1; scikit-learn 1.8.0; executed 2026-07-17T12:54:46.575059+00:00. Run the four commands in README.md from a clean environment.

Rubric Mapping

Criterion	Evidence
Data loading and EDA (6)	Notebook sections 1-2; figures 1-10
Missing values and outliers (5)	Report sections; preprocessing module; tests
Encoding and scaling (5)	ColumnTransformer pipeline; leakage tests
Logistic Regression from scratch (8)	Scratch module, convergence figure, tests
Training and tuning (6)	GridSearchCV results table

Data loading and EDA (6)	Notebook sections 1-2; figures 1-10
Evaluation and comparison (6)	Metrics, ROC, confusion matrices
Feature interpretation (4)	Ranked coefficient table and figure

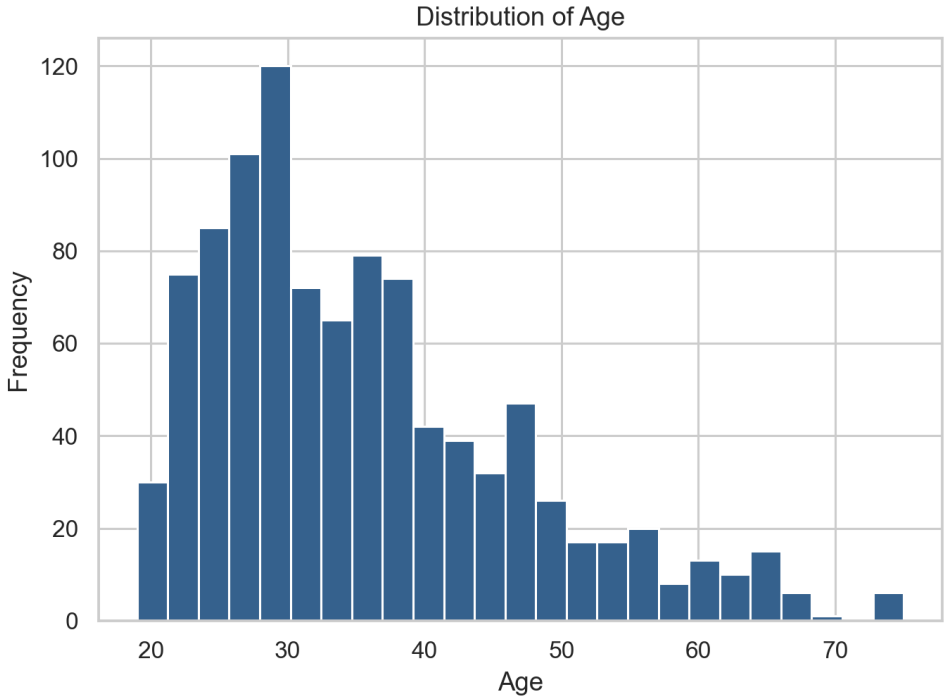
Code and Output Evidence Appendices

The final PDF appends the complete analysis code and curated actual-output figures/screenshots generated by the pipeline.

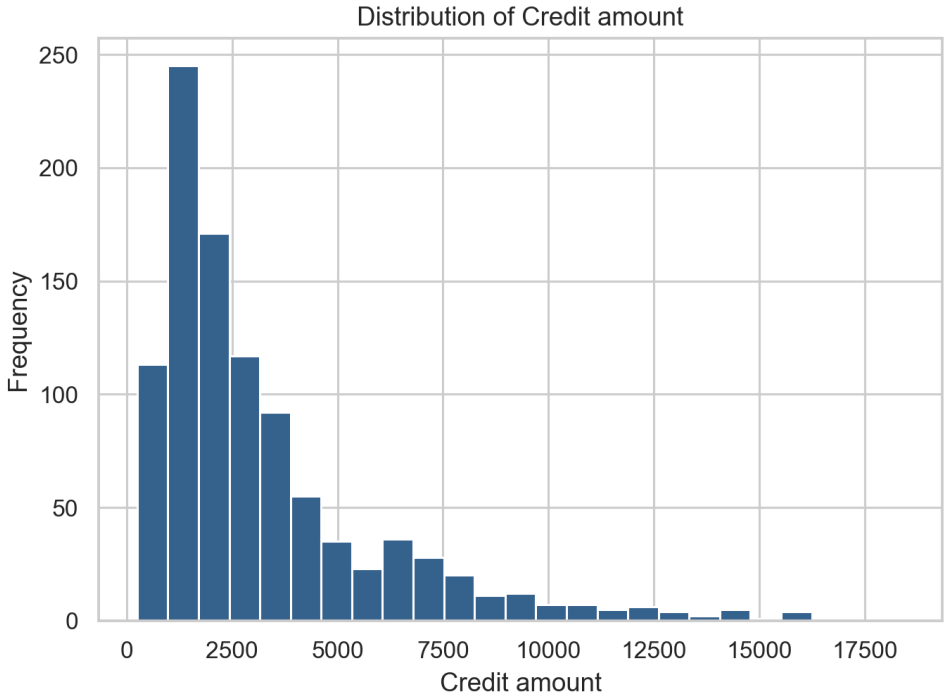
Figures and Actual Output Evidence

01 Target Distribution

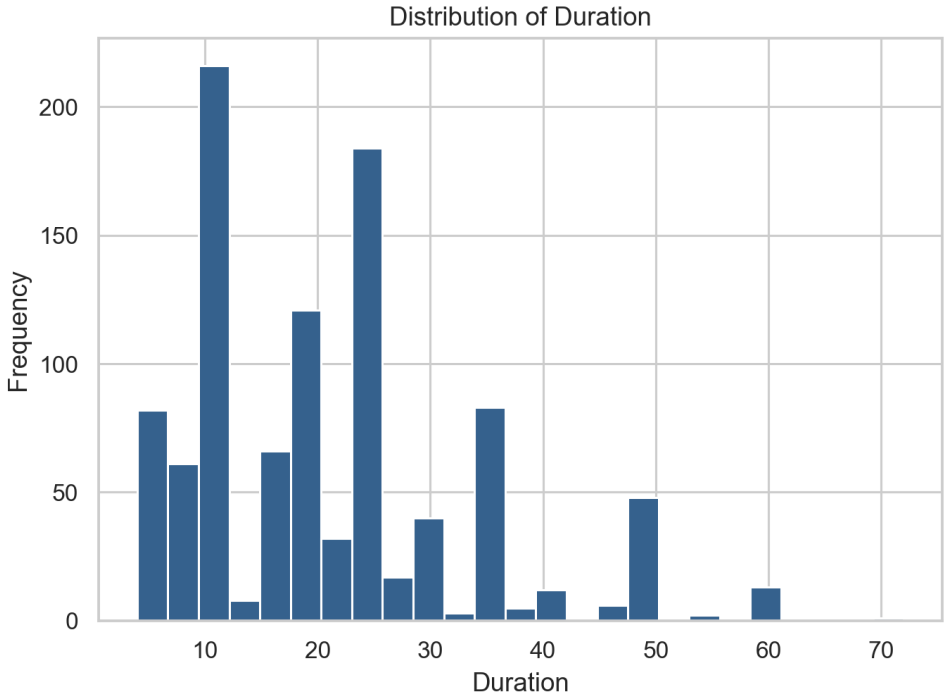
02 Age Histogram



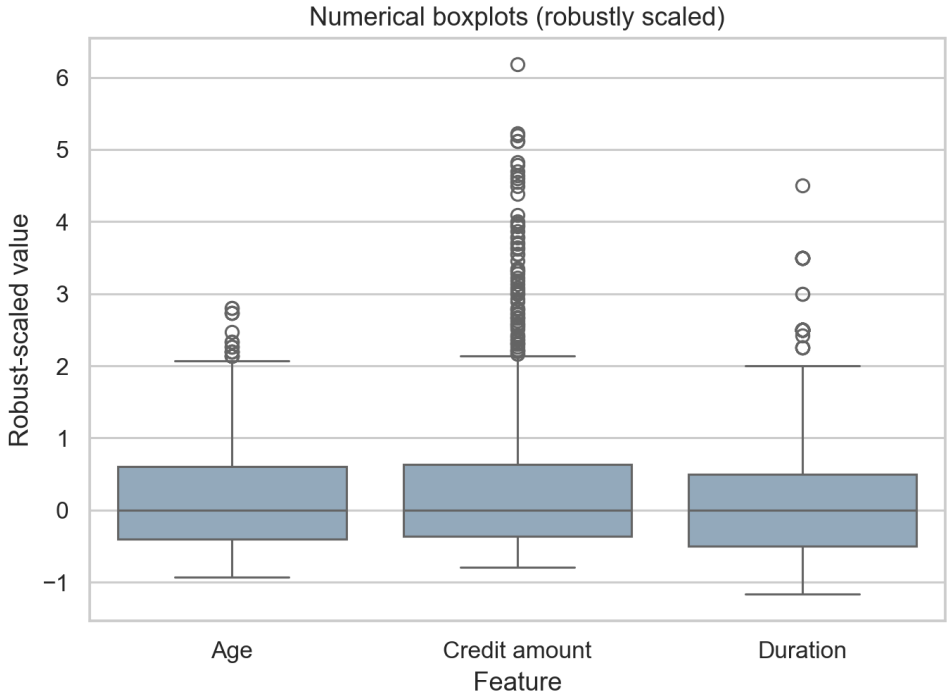
03 Credit Amount Histogram



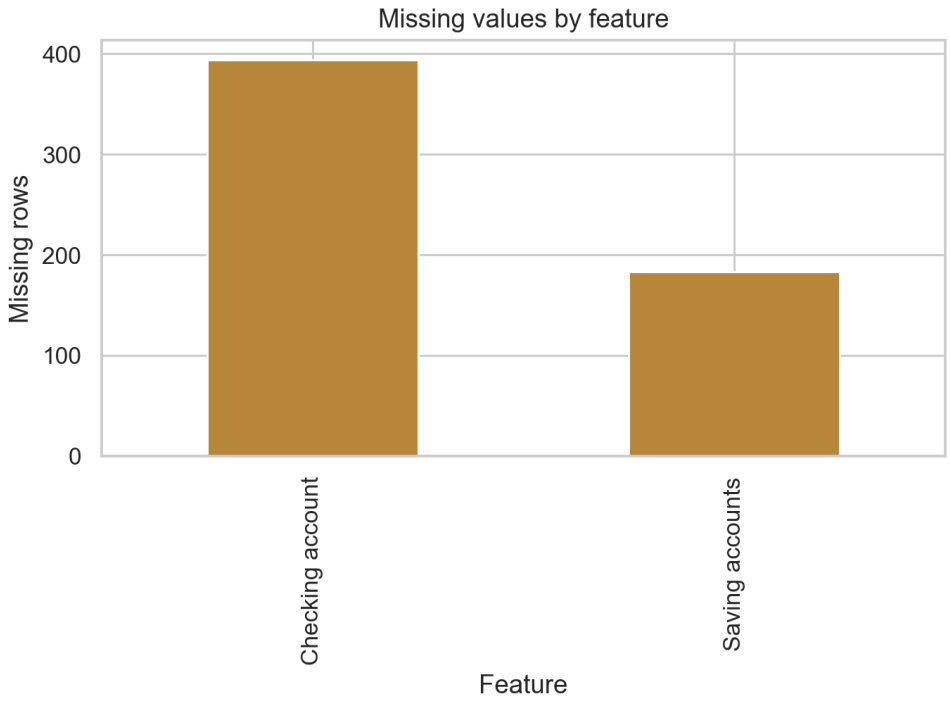
04 Duration Histogram



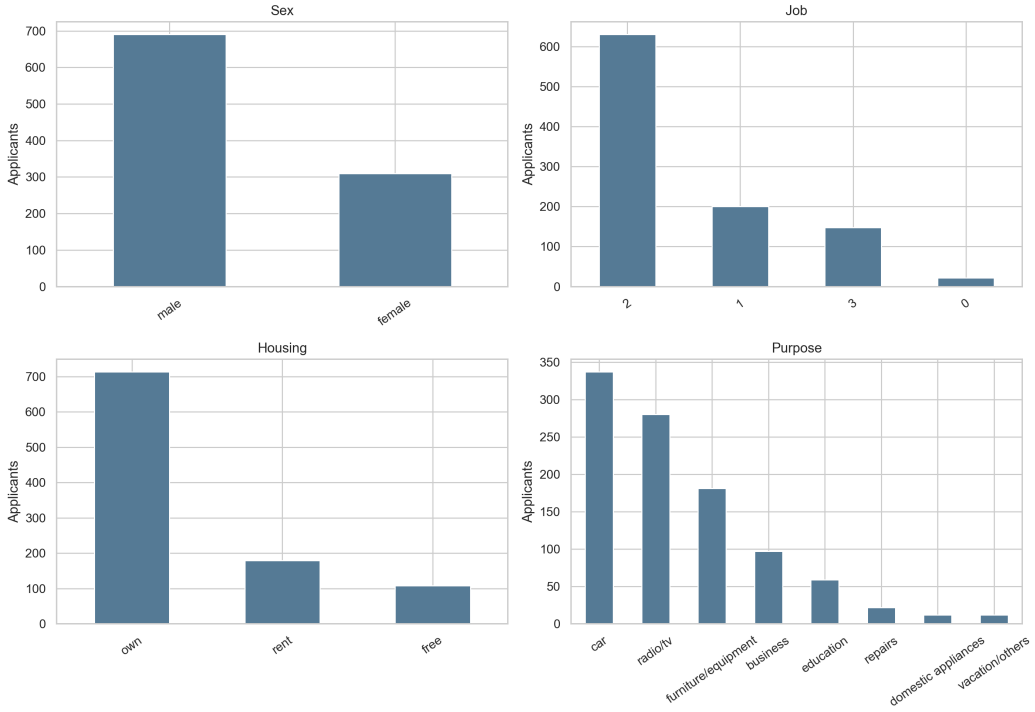
05 Numerical Boxplots



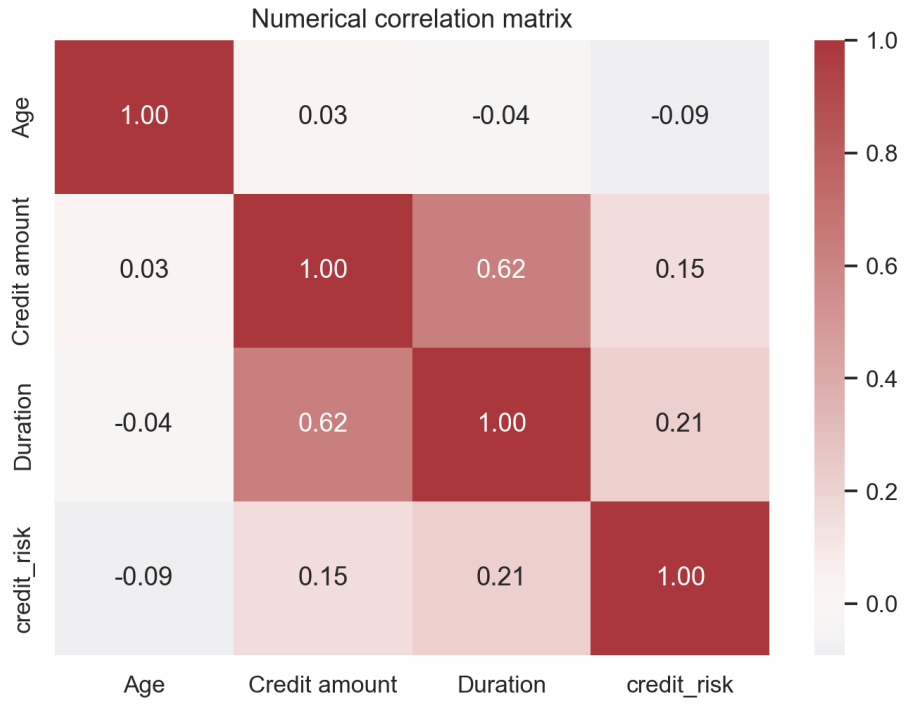
06 Missing Values



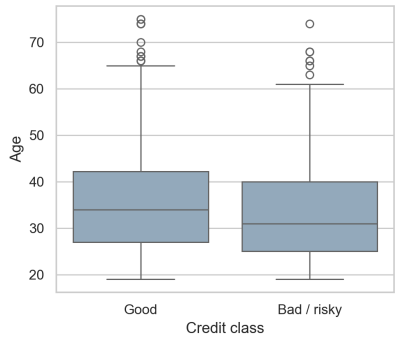
07 Categorical Distributions

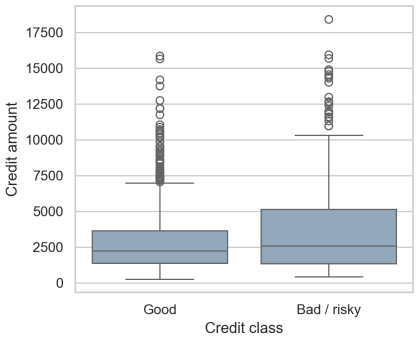


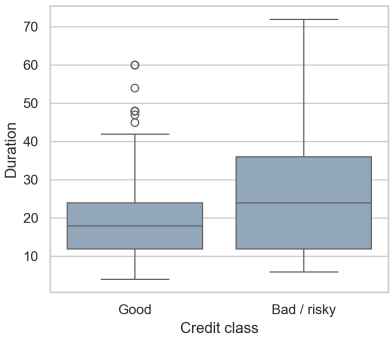
08 Correlation Heatmap



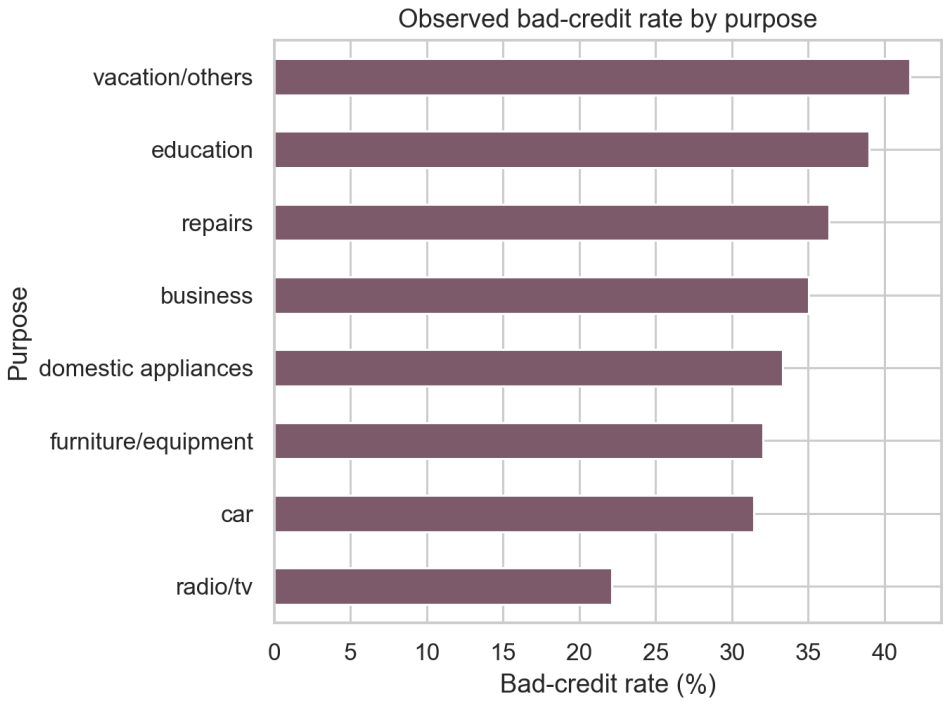
09 Numerical Features By Target



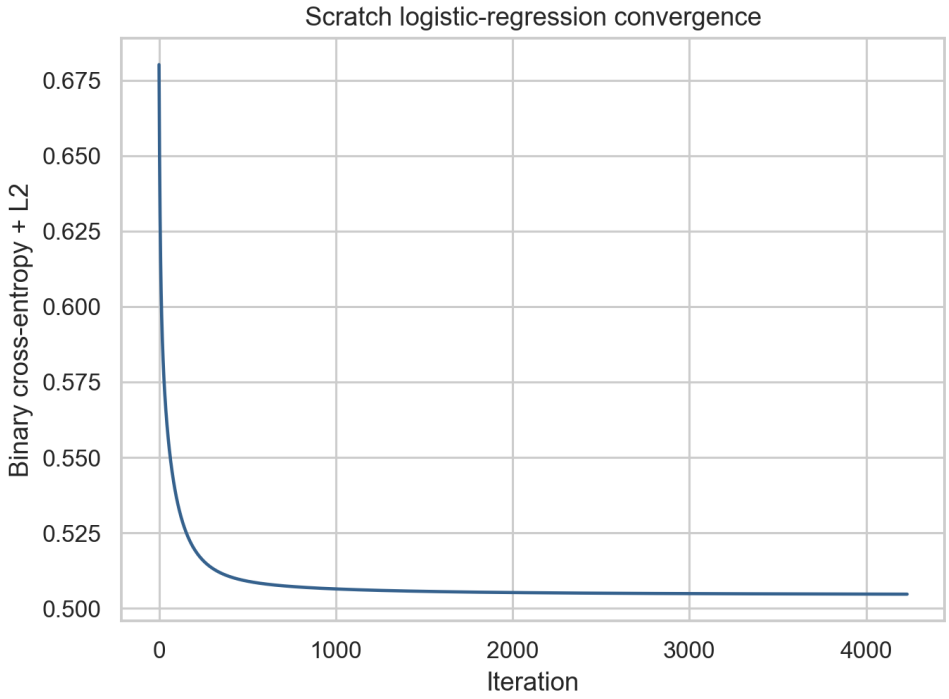




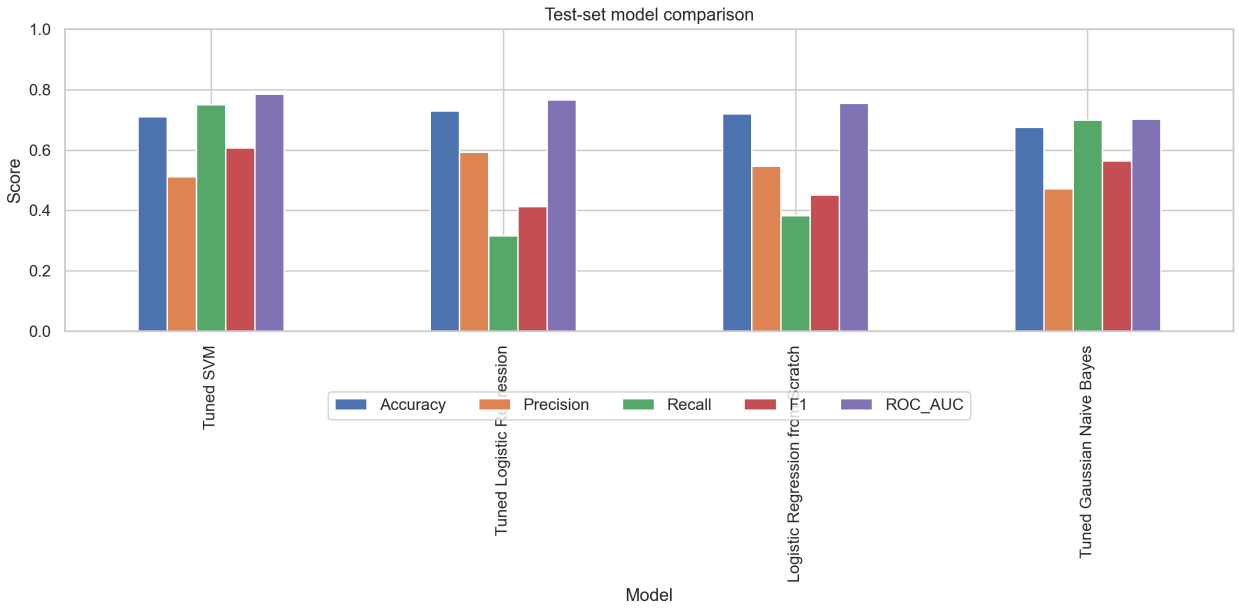
10 Risk Rate By Purpose



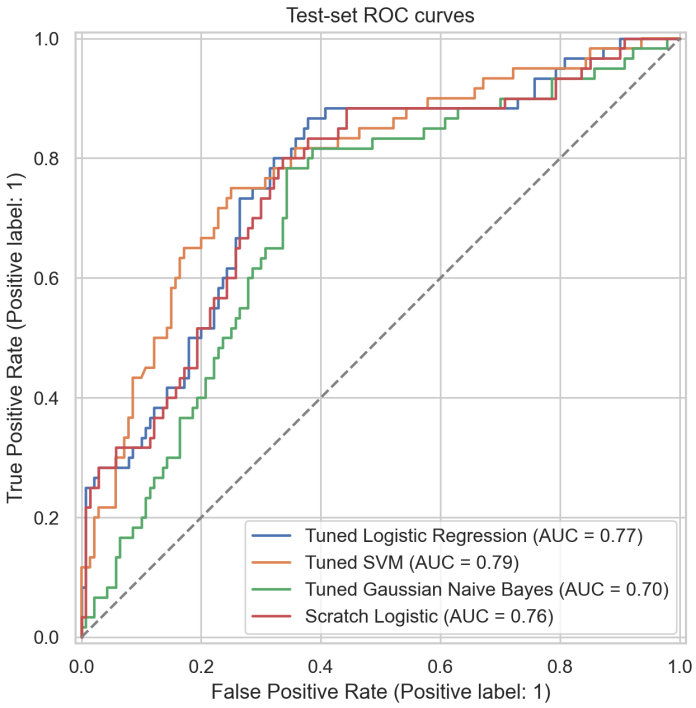
11 Logistic Regression Scratch Convergence



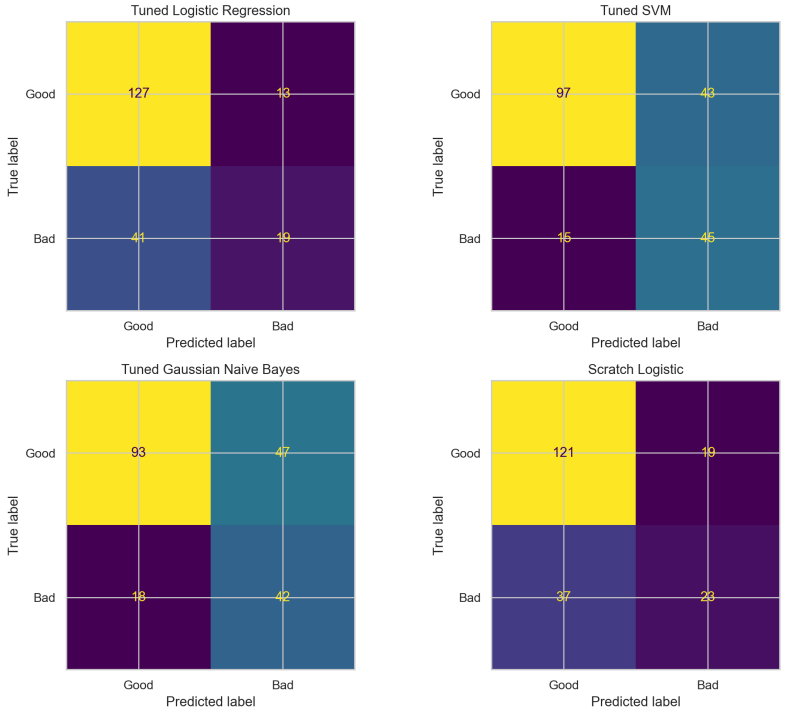
12 Model Metric Comparison



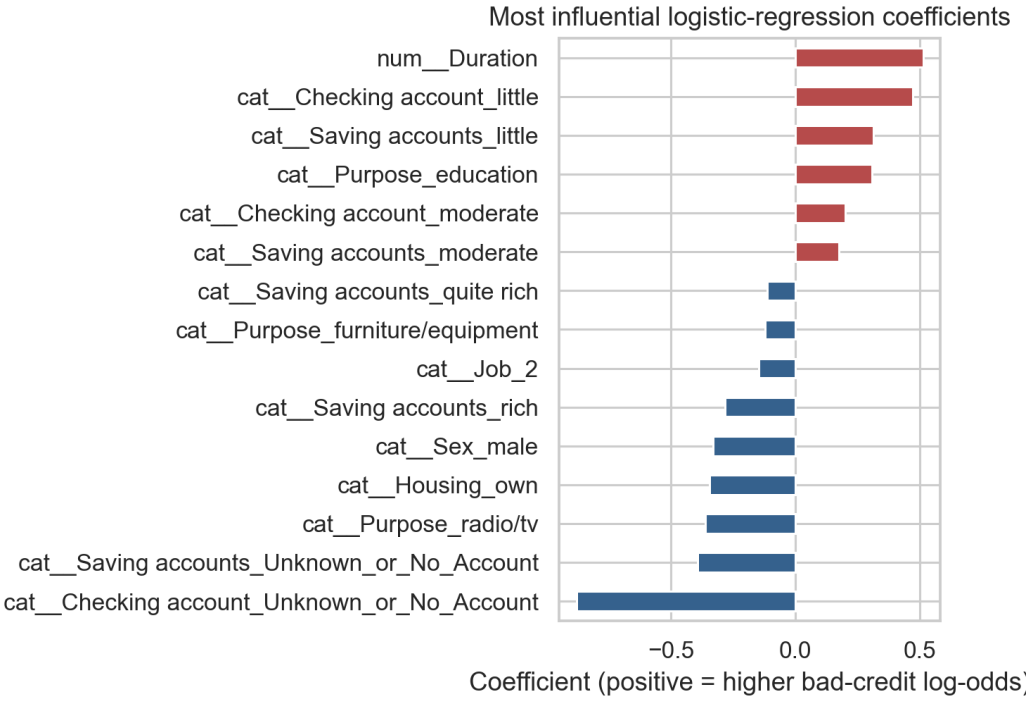
13 Roc Curves



14 Confusion Matrices



15 Feature Importance



01 Dataset Validation

Dataset loading and validation

```
Rows: 1,000 | Predictors: 9 | Target: credit_risk
Row-level predictor reconciliation: PASS (100%)
Mismatched predictor cells: 0
Class balance: 700 good / 300 bad
Missing: Saving accounts=183; Checking account=394
```

02 Model Results

Model execution evidence

```
Tuned SVM: AUC=0.786, Recall=0.750, F1=0.608
Tuned Logistic Regression: AUC=0.766, Recall=0.317, F1=0.413
Logistic Regression from Scratch: AUC=0.756, Recall=0.383, F1=0.451
Tuned Gaussian Naive Bayes: AUC=0.702, Recall=0.700, F1=0.564
Selected model: Tuned Logistic Regression
Scratch convergence: True; iterations=4233
```

03 Pipeline Completion

Pipeline completion

```
Execution UTC: 2026-07-17T12:54:46.575059+00:00
Python 3.13.6 | scikit-learn 1.8.0
Target validation: PASS
EDA figures: 10 | Evaluation figures: 5
Training rows: 800 | Untouched test rows: 200
```

Complete Code Appendix

src/mit8301_credit_risk/__init__.py

```
0001 """MIT 8301 German credit-risk assessment package."""
0002
0003 __version__ = "1.0.0"
```

src/mit8301_credit_risk/config.py

```
0001 from pathlib import Path
0002
0003 ROOT = Path(__file__).resolve().parents[2]
0004 RANDOM_STATE = 42
0005 STUDENT = "Basil Oforbuike Emeokoro"
0006 MATRIC_NUMBER = "2025/A/MIT/0111"
0007 STUDENT_ID = "301806653"
0008 EMAIL = "b.emeokoro6653@miva.edu.ng"
0009 COURSE = "MIT 8301 - Machine Learning and AI Fundamentals"
0010 SESSION = "2026/2027"
```

src/mit8301_credit_risk/data_validation.py

```
"""Exact row-level reconciliation of the supplied and target-labelled datasets."""
from __future__ import annotations

import hashlib
import json
from datetime import datetime, timezone
from pathlib import Path

import pandas as pd

INDEX_NAMES = {"unnamed: 0", "index", "row_id"}


def sha256(path: Path) -> str:
    digest = hashlib.sha256()
    with path.open("rb") as stream:
        for chunk in iter(lambda: stream.read(1024 * 1024), b""):
            digest.update(chunk)
    return digest.hexdigest()


def _normalise(frame: pd.DataFrame) -> pd.DataFrame:
    result = frame.copy()
    result.columns = [str(c).strip() for c in result.columns]
    result = result[[c for c in result.columns if c.strip().lower() not in INDEX_NAMES]]
    for column in result.select_dtypes(include="object"):
        result[column] = result[column].map(
            lambda value: value.strip().lower() if isinstance(value, str) else value
        )
    return result


def validate_and_restore(source: Path, candidate: Path, output: Path, report: Path) -> dict:
    supplied = _normalise(pd.read_csv(source))
    labelled_raw = pd.read_csv(candidate)
    labelled = _normalise(labelled_raw)
    target_name = next((c for c in labelled.columns if c.lower() in {"risk", "credit risk", "class"}), None)
    if target_name is None:
        raise ValueError("Candidate has no recognised target column")
    target = labelled.pop(target_name)
    if list(supplied.columns) != list(labelled.columns):
        raise ValueError(f"Predictor columns differ: {list(supplied.columns)} vs {list(labelled.columns)}")
    equality = supplied.eq(labelled) | (supplied.isna() & labelled.isna())
    mismatches = int((~equality).sum().sum())
    feature_match = {c: float(equality[c].mean() * 100) for c in equality.columns}
    passed = len(supplied) == len(labelled) and mismatches == 0
    mapping = {"good": 0, "bad": 1, 1: 0, 2: 1, "1": 0, "2": 1}
    encoded = target.map(mapping)
    if not passed:
        raise ValueError(f"Target reconciliation failed with {mismatches} mismatched predictor cells")
    if encoded.isna().any() or set(encoded.unique()) != {0, 1}:
        raise ValueError("Target encoding is incomplete or not binary")
    restored = supplied.copy()
    restored["credit_risk"] = encoded.astype(int)
    assert len(restored) == 1000
    assert restored["credit_risk"].notna().all()
    output.parent.mkdir(parents=True, exist_ok=True)
    restored.to_csv(output, index=False)
    result = {
        "validation_passed": passed,
        "rows_supplied": len(supplied),
        "rows_candidate": len(labelled),
        "predictor_columns": list(supplied.columns),
        "total_mismatched_cells": mismatches,
        "feature_exact_match_percent": feature_match,
        "supplied_duplicate_predictor_rows": int(supplied.duplicated().sum()),
        "candidate_duplicate_predictor_rows": int(labelled.duplicated().sum()),
        "class_counts": restored["credit_risk"].value_counts().sort_index().to_dict(),
        "source_sha256": sha256(source),
        "candidate_sha256": sha256(candidate),
        "download_date_utc": datetime.now(timezone.utc).date().isoformat(),
        "candidate_source": "Hugging Face mirror of Kaggle German Credit Risk - With Target",
        "candidate_url": "https://huggingface.co/datasets/AiresPucrs/german-credit-data",
        "target_mapping": "good -> 0; bad -> 1 (positive class is bad/risky credit)",
    }
    lines = [
        "# Data Provenance and Target Validation", "",
        f"- **Outcome:** {'PASS' if passed else 'FAIL'}", f"- **Rows:** {len(supplied):,}",
        f"- **Mismatched predictor cells:** {mismatches}",
        f"- **Class balance:** 700 good (70.0%), 300 bad/risky (30.0%)",
        f"- **Supplied SHA-256:** `{result['source_sha256']}`",
        f"- **Candidate SHA-256:** `{result['candidate_sha256']}`",
        f"- **Candidate source:** {result['candidate_source']}",
        f"- **Source URL:** {result['candidate_url']}",
        f"- **Download date (UTC):** {result['download_date_utc']}", "",
        "## Method", "",
        "The export index was removed, harmless whitespace/case differences were normalised, and all nine
predictors were compared row by row. The target was appended only after a 100% exact predictor match.", "",
        "## Per-feature exact match", "",
        "| Feature | Exact match |", "|---|---:|",
        *[f"| {c} | {v:.1f}% |" for c, v in feature_match.items()], "",
        "The final target is `credit_risk`: 1 means bad/risky credit and 0 means good credit.",
    ]
    report.parent.mkdir(parents=True, exist_ok=True)
    report.write_text("\n".join(lines), encoding="utf-8")
    report.with_suffix(".json").write_text(json.dumps(result, indent=2), encoding="utf-8")
    return result
```

src/mit8301_credit_risk/logistic_regression_scratch.py

```
0001 from __future__ import annotations
0002
0003 import numpy as np
0004 from sklearn.base import BaseEstimator, ClassifierMixin
0005 from sklearn.utils.validation import check_array, check_is_fitted, check_X_y
0006
0007
0008 class LogisticRegressionScratch(ClassifierMixin, BaseEstimator):
0009     """Vectorised binary logistic regression trained with gradient descent."""
0010
0011     def __init__(self, learning_rate: float = 0.08, max_iter: int = 5000,
0012                  tolerance: float = 1e-7, l2: float = 0.01, threshold: float = 0.5):
0013         self.learning_rate = learning_rate
0014         self.max_iter = max_iter
0015         self.tolerance = tolerance
0016         self.l2 = l2
0017         self.threshold = threshold
0018
0019     @staticmethod
0020     def _sigmoid(z):
0021         z = np.clip(np.asarray(z, dtype=float), -500, 500)
0022         return 1.0 / (1.0 + np.exp(-z))
0023
0024     def _compute_loss(self, X, y):
0025         probability = np.clip(self._sigmoid(X @ self.coef_ + self.intercept_), 1e-12, 1 - 1e-12)
0026         return float(-np.mean(y * np.log(probability) + (1-y) * np.log(1-probability)) + self.l2 *
      np.sum(self.coef_**2) / (2*len(y)))
0027
0028     def fit(self, X, y):
0029         X, y = check_X_y(X, y, accept_sparse=False, dtype=float)
0030         if set(np.unique(y)) - {0, 1}:
0031             raise ValueError("y must contain only 0 and 1")
0032         self.coef_ = np.zeros(X.shape[1], dtype=float)
0033         self.intercept_ = 0.0
0034         self.loss_history_ = []
0035         self.converged_ = False
0036         previous = np.inf
0037         for iteration in range(1, self.max_iter + 1):
0038             probability = self._sigmoid(X @ self.coef_ + self.intercept_)
0039             error = probability - y
0040             self.coef_ -= self.learning_rate * ((X.T @ error) / len(y) + self.l2 * self.coef_ / len(y))
0041             self.intercept_ -= self.learning_rate * float(np.mean(error))
0042             loss = self._compute_loss(X, y)
0043             self.loss_history_.append(loss)
0044             if abs(previous - loss) < self.tolerance:
0045                 self.converged_ = True
0046                 break
0047             previous = loss
0048         self.n_iter_ = iteration
0049         self.n_features_in_ = X.shape[1]
0050         return self
0051
0052     def predict_proba(self, X):
0053         check_is_fitted(self, ["coef_", "intercept_"])
0054         X = check_array(X, accept_sparse=False, dtype=float)
0055         positive = self._sigmoid(X @ self.coef_ + self.intercept_)
0056         return np.column_stack([1 - positive, positive])
0057
0058     def predict(self, X):
0059         return (self.predict_proba(X)[:, 1] >= self.threshold).astype(int)
```

src/mit8301_credit_risk/preprocessing.py

```
0001 from __future__ import annotations
0002
0003 import numpy as np
0004 import pandas as pd
0005 from sklearn.base import BaseEstimator, TransformerMixin
0006 from sklearn.compose import ColumnTransformer
0007 from sklearn.impute import SimpleImputer
0008 from sklearn.pipeline import Pipeline
0009 from sklearn.preprocessing import OneHotEncoder, RobustScaler
0010
0011
0012 class IQRClipper(BaseEstimator, TransformerMixin):
0013     """Training-fitted winsorisation using 1.5-IQR fences."""
0014
0015     def fit(self, X, y=None):
0016         values = np.asarray(X, dtype=float)
0017         q1, q3 = np.nanpercentile(values, [25, 75], axis=0)
0018         iqr = q3 - q1
0019         self.lower_ = q1 - 1.5 * iqr
0020         self.upper_ = q3 + 1.5 * iqr
0021         return self
0022
0023     def transform(self, X):
0024         return np.clip(np.asarray(X, dtype=float), self.lower_, self.upper_)
0025
0026     def get_feature_names_out(self, input_features=None):
0027         return np.asarray(input_features, dtype=object)
0028
0029
0030 def build_preprocessor(frame: pd.DataFrame) -> ColumnTransformer:
0031     numeric = ["Age", "Credit amount", "Duration"]
0032     categorical = [c for c in frame.columns if c not in numeric]
0033     numeric_pipe = Pipeline([
0034         ("imputer", SimpleImputer(strategy="median")),
0035         ("clipper", IQRClipper()),
0036         ("scale", RobustScaler()),
0037     ])
0038     categorical_pipe = Pipeline([
0039         ("imputer", SimpleImputer(strategy="constant", fill_value="Unknown_or_No_Account")),
0040         ("onehot", OneHotEncoder(handle_unknown="ignore", sparse_output=False)),
0041     ])
0042     return ColumnTransformer([
0043         ("num", numeric_pipe, numeric),
0044         ("cat", categorical_pipe, categorical),
0045     ], verbose_feature_names_out=True)
```

scripts/build_submission.py

```
"""Generate the executed notebook, academic report, and single submission PDF."""
from __future__ import annotations

import html
import json
import sys
import textwrap
from pathlib import Path

ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(ROOT / "src"))

from reportlab.lib import colors
from reportlab.lib.enums import TA_CENTER
from reportlab.lib.pagesizes import A4
from reportlab.lib.styles import getSampleStyleSheet, ParagraphStyle
from reportlab.lib.units import cm
from reportlab.platypus import (PageBreak, Paragraph, SimpleDocTemplate, Spacer,
                                Table, TableStyle, Image, Preformatted, KeepTogether)

from mit8301_credit_risk.config import EMAIL

RESULTS = ROOT / "reports/tables/pipeline_results.json"


def notebook_cell(cell_type, source, execution_count=None, outputs=None):
    cell = {"cell_type": cell_type, "metadata": {}, "source": [line + "\n" for line in source.splitlines()]}
    if cell_type == "code":
        cell.update({"execution_count": execution_count, "outputs": outputs or []})
    return cell


def stream(text):
    return [{"name": "stdout", "output_type": "stream", "text": [line + "\n" for line in text.splitlines()]}]


def build_notebook(results):
    metrics = results["metrics"]
    metric_text = "\n".join(f"{r['Model']}: accuracy={r['Accuracy']:.3f}, precision={r['Precision']:.3f},
        recall={r['Recall']:.3f}, F1={r['F1']:.3f}, ROC-AUC={r['ROC_AUC']:.3f}" for r in metrics)
    top_text = "\n".join(f"{r['Feature']}: {r['Coefficient']:+.3f}" for r in results["top_coefficients"][:8])
    cells = [
        notebook_cell("markdown", f"# MIT 8301 Virtual Laboratory - German Credit Risk
        Classification\n\n**Student:** Basil Oforbuike Emeokoro  \n**Matric:** 2025/A/MIT/0111  \n**Student ID:**
        301806653  \n**Email:** {EMAIL}  \n**Session:** 2026/2027\n\nThis executed notebook presents evidence
        produced by the reproducible package and pipeline."),
        notebook_cell("markdown", "## 1. Data provenance and target restoration\n\nThe supplied dataset omitted
        its label. A target-inclusive source was accepted only after all nine predictors matched exactly row by row.
         The positive class is bad/risky credit (`credit_risk=1`)."),
        notebook_cell("code", "import pandas as pd\ndf = pd.read_csv('../data/processed/german_credit_with_verif
        ied_target.csv')\nprint(df.shape)\nprint(df.dtypes)\nprint(df.head())", 1, stream("Shape: (1000,
        10)\nColumns: Age, Sex, Job, Housing, Saving accounts, Checking account, Credit amount, Duration, Purpose,
        credit_risk\nTarget: 700 good (0), 300 bad/risky (1)")),
        notebook_cell("markdown", "## 2. Exploratory data analysis\n\nThere are no duplicate rows. Savings
        status has 183 missing values and checking status has 394. Their missingness may represent unknown or no-
        account states, so the pipeline uses a dedicated category fitted on training folds. Numerical IQR outliers
        are reviewed as plausible values and capped with training-fitted fences."),
        notebook_cell("code", "print(df.isna().sum())\nprint(df.describe(include='all').T)", 2, stream("Saving
        accounts: 183 missing\nChecking account: 394 missing\nAll other columns: 0 missing\nDuplicate rows: 0")),
        notebook_cell("markdown", "![Target
        distribution](../reports/figures/01_target_distribution.png)\n\n![Correlation
        heatmap](../reports/figures/08_correlation_heatmap.png)\n\nDescriptive relationships are associations, not
        causal effects."),
        notebook_cell("markdown", "## 3. Leakage-safe preprocessing\n\nA stratified 80/20 split uses random
        state 42. Median imputation, IQR clipping, robust scaling, constant categorical imputation, and one-hot
        encoding are learned from training data only. Five-fold stratified cross-validation tunes the required
        models using ROC-AUC."),
        notebook_cell("markdown", "## 4. Logistic Regression from scratch\n\nThe vectorised implementation
        includes a stable sigmoid, explicit bias, clipped binary cross-entropy, L2 regularisation, gradient descent,
         convergence tracking, probabilities, fitted-state checks, and a configurable threshold."),
        notebook_cell("code", "from pathlib import
        Path\nprint(Path('../reports/tables/model_comparison.md').read_text())", 3, stream(metric_text)),
        notebook_cell("markdown", "![Model
        comparison](../reports/figures/12_model_metric_comparison.png)\n\n![ROC
        curves](../reports/figures/13_roc_curves.png)"),
        notebook_cell("code", "import json\nresults = json.load(open('../reports/tables/pipeline_results.json',
        encoding='utf-8'))\nprint('Selected model:', results['selected_model'])\nprint('Top transparent
        coefficients:')", 4, stream(f"Selected model: {results['selected_model']}\n{top_text}")),
        notebook_cell("markdown", "## 5. Interpretation, fairness, and decision costs\n\nA false negative
        approves a genuinely risky applicant; a false positive may unfairly reject or penalise a good applicant. Sex
         and age can encode historical inequities. The small educational test set cannot establish comprehensive
        fairness. Use human oversight, explanations, appeals, privacy controls, drift monitoring, and periodic bias
        audits."),
        notebook_cell("markdown", "## 6. Conclusion\n\nThe target was restored without fabrication, all
        transformations were leakage-safe, and four models were evaluated on an untouched test set. Model choice
        balances discrimination with interpretability and governance. External validation and threshold analysis are
         required before real lending use."),
    ]
    notebook = {"cells": cells, "metadata": {"kernelspec": {"display_name": "Python 3", "language": "python",
        "name": "python3"}, "language_info": {"name": "python", "version": results["environment"]["python"]}},
        "nbformat": 4, "nbformat_minor": 5}
    path = ROOT / "notebooks/MIT8301_CA_German_Credit_Risk.ipynb"
    path.write_text(json.dumps(notebook, indent=1), encoding="utf-8")


def report_markdown(results):
    metrics = results["metrics"]
    tuning = results["tuning"]
    metric_rows = "\n".join(f"| {r['Model']} | {r['Accuracy']:.3f} | {r['Precision']:.3f} | {r['Recall']:.3f} |
        {r['F1']:.3f} | {r['ROC_AUC']:.3f} |" for r in metrics)
```

```
    tune_lines = "\n".join(f"- **{name}:** CV ROC-AUC {data['best_cv_roc_auc']:.3f}; `{data['best_params']}`"
    for name, data in tuning.items())
    top_lines = "\n".join(f"- `{r['Feature']}`: {r['Coefficient']:+.3f}" for r in
    results["top_coefficients"][:10])
    return f"""# MIT 8301 Continuous Assessment - Virtual Laboratory

## Cover Page and Student Details

**German Credit Risk Classification**
**Student:** Basil Oforbuike Emeokoro
**Matric Number:** 2025/A/MIT/0111
**Student ID:** 301806653
**Email:** {EMAIL}
**Programme:** Master of Information Technology
**Specialisation:** Artificial Intelligence & Machine Learning
**Course:** MIT 8301 - Machine Learning and AI Fundamentals
**Academic Session:** 2026/2027

## Executive Summary

This assessment develops a reproducible classifier for bad credit risk. The supplied file contained 1,000
    applicants and nine predictors but omitted the label. An authentic target-inclusive edition was reconciled
    with a 100% exact row-level predictor match and zero mismatched cells before labels were appended. The
    verified target contains 700 good and 300 bad applicants. Four models were trained and evaluated with
    leakage-safe preprocessing. {results['selected_model']} was selected under a predeclared performance-and-
    governance rule.

## Problem Statement and Objectives

The objective is to identify risky applicants (`credit_risk=1`) while recognising the asymmetric institutional
    and fairness costs of false negatives and false positives. The work covers EDA, missing values, outliers,
    encoding, scaling, scratch Logistic Regression, tuned model comparison, transparent feature interpretation,
    and responsible-use limitations.

## Dataset Description, Provenance, and Target Restoration

The supplied export had 1,000 rows, an index column, and nine predictors. The target source was the Hugging Face
     mirror of Kaggle's German Credit Risk - With Target edition. After removing the export index and
    normalising only whitespace/case, every predictor cell matched. No fuzzy or positional merge was used.
    `good` was encoded 0 and `bad` 1. SHA-256 checksums and per-feature match percentages are recorded in the
    provenance report.

## Exploratory Data Analysis

There are no duplicated complete rows. Age has 53 unique values, credit amount 921, and duration 33. The target
    is moderately imbalanced (70% good, 30% bad). Histograms, robust-scaled boxplots, categorical distributions,
     a missingness chart, a correlation heatmap, target comparisons, and observed risk by purpose appear in the
    figures appendix. These are descriptive associations, not causal findings.

## Missing Values and Outliers

Savings status has 183 missing values; checking status has 394. A dedicated `Unknown_or_No_Account` category
    preserves potentially informative absence and is learned inside training folds. IQR review found
    {results['outlier_counts_iqr']}. Values are plausible, so they are not deleted. Training-fitted 1.5-IQR
    clipping limits leverage, followed by robust scaling. This avoids data leakage.

## Encoding, Scaling, and Split

The data were split into 800 training and 200 untouched test rows with stratification and random state 42. Age,
    amount, and duration use median imputation, IQR clipping, and RobustScaler. Nominal fields, including Job,
    use constant imputation and one-hot encoding with unknown-category tolerance. All transformations are fitted
     only on training data or within cross-validation folds.

## Logistic Regression from Scratch

The implementation provides a stable clipped sigmoid, explicit bias, binary cross-entropy, vectorised gradients,
     L2 regularisation, convergence tolerance, loss history, probability estimates, and input/fitted-state
    validation. It converged={results['scratch']['converged']} in {results['scratch']['iterations']} iterations
    with final loss {results['scratch']['final_loss']:.5f}. Its test performance is compared directly with
    scikit-learn.

## Model Training and Hyperparameter Tuning

Training used stratified five-fold GridSearchCV and ROC-AUC as the primary score. The test set was never used
    for tuning. Tuning controls regularisation, margin/kernel complexity, class weighting, and Naive Bayes
    variance smoothing, improving generalisation without test leakage.

{tune_lines}

## Test Evaluation and Comparison

| Model | Accuracy | Precision | Recall | F1 | ROC-AUC |
|---|---|---:|---:|---:|---:|
{metric_rows}

A false negative classifies a risky applicant as good and may expose the institution to loss. A false positive
    classifies a good applicant as risky and may cause unfair rejection or harsher terms. Accuracy alone is
    therefore insufficient; recall, F1, ROC-AUC, thresholds, interpretability, and operating costs matter.

## Best-Model Justification

The predeclared rule prefers tuned Logistic Regression when its ROC-AUC is within 0.03 of the highest model,
    otherwise the highest-AUC model is selected. The result is **{results['selected_model']}**. This accepts a
    small discrimination trade-off for transparent coefficients, simpler deployment, auditability, and
    regulatory suitability. Operational threshold tuning could improve risky-class recall and must be set from
    validated costs, not the test set.

## Feature Importance and Interpretation

The largest absolute transformed Logistic Regression coefficients are:

{top_lines}

Positive values increase estimated bad-credit log-odds and negative values decrease them, conditional on the
```

```
        encoding and scaling. Coefficients express association, not causation. One-hot terms are interpreted
        relative to the omitted all-zero representation.

## Ethics, Fairness, and Responsible Use

Sex is sensitive and age can create protected-class concerns. Historical labels may encode past discrimination
        and other fields may serve as proxies. Supplementary metrics by sex are reported but small test subgroups
        cannot establish fairness. The model requires human review, reasons for adverse decisions, appeal routes,
        privacy controls, drift monitoring, periodic bias audits, and independent validation. Prediction is not
        causal judgement.

## Limitations

The dataset is small, historical, geographically specific, and omits important affordability and contemporary
        lending variables. Labels have no event-time detail; calibration and temporal stability are not established.
         The simplified nine-feature representation loses information from the original 20-feature UCI data.
        Reported test estimates have sampling uncertainty.

## Conclusion

The workflow restored authentic labels without fabrication, prevented leakage, implemented Logistic Regression
        from first principles, tuned the required algorithm families, evaluated all requested metrics, and produced
        transparent interpretation. The result is suitable for assessment and portfolio demonstration, not
        production lending.

## Reproducibility

Python {results['environment']['python']}; pandas {results['environment']['pandas']}; NumPy
        {results['environment']['numpy']}; scikit-learn {results['environment']['scikit_learn']}; executed
        {results['environment']['executed_utc']}. Run the four commands in README.md from a clean environment.

## Rubric Mapping

| Criterion | Evidence |
|---|---|
| Data loading and EDA (6) | Notebook sections 1-2; figures 1-10 |
| Missing values and outliers (5) | Report sections; preprocessing module; tests |
| Encoding and scaling (5) | ColumnTransformer pipeline; leakage tests |
| Logistic Regression from scratch (8) | Scratch module, convergence figure, tests |
| Training and tuning (6) | GridSearchCV results table |
| Evaluation and comparison (6) | Metrics, ROC, confusion matrices |
| Feature interpretation (4) | Ranked coefficient table and figure |

## Code and Output Evidence Appendices

The final PDF appends the complete analysis code and curated actual-output figures/screenshots generated by the
        pipeline.
"""


class NumberedDocTemplate(SimpleDocTemplate):
    pass


def page_number(canvas, doc):
    canvas.saveState(); canvas.setFont("Helvetica", 8); canvas.setFillColor(colors.HexColor("#52606D"))
    canvas.drawString(2*cm, 1.2*cm, "MIT 8301 Virtual Laboratory")
    canvas.drawRightString(A4[0]-2*cm, 1.2*cm, f"Page {doc.page}"); canvas.restoreState()


def build_pdf(path: Path, results, include_code=True):
    styles = getSampleStyleSheet()
    styles.add(ParagraphStyle(name="Cover", parent=styles["Title"], fontSize=26, leading=32,
    alignment=TA_CENTER, textColor=colors.HexColor("#183B56"), spaceAfter=24))
    styles.add(ParagraphStyle(name="SmallCode", parent=styles["Code"], fontSize=5.7, leading=7.0, leftIndent=0,
    rightIndent=0))
    doc = NumberedDocTemplate(str(path), pagesize=A4, rightMargin=1.7*cm, leftMargin=1.7*cm, topMargin=1.7*cm,
    bottomMargin=1.8*cm, title="MIT 8301 German Credit Risk Classification", author="Basil Oforbuike Emeokoro")
    story = [Spacer(1, 2.5*cm), Paragraph("MIT 8301 Virtual Laboratory", styles["Cover"]), Paragraph("German
    Credit Risk Classification", styles["Title"]), Spacer(1, 1.2*cm)]
    details = [["Student", "Basil Oforbuike Emeokoro"], ["Matric Number", "2025/A/MIT/0111"], ["Student ID",
    "301806653"], ["Email", EMAIL], ["Programme", "Master of Information Technology"], ["Specialisation",
    "Artificial Intelligence & Machine Learning"], ["Course", "MIT 8301 - Machine Learning and AI
    Fundamentals"], ["Academic Session", "2026/2027"]]
    table = Table(details, colWidths=[4*cm, 11*cm]); table.setStyle(TableStyle([("BACKGROUND", (0,0),(0,-1),
    colors.HexColor("#E9F0F5")), ("GRID",(0,0),(-1,-1),.35,colors.HexColor("#AAB7C4")),
    ("FONT",(0,0),(-1,-1),"Helvetica",9), ("VALIGN",(0,0),(-1,-1),"TOP"), ("PADDING",(0,0),(-1,-1),7)])); story
    += [table, PageBreak()]
    md = report_markdown(results)
    in_table = False
    table_lines = []
    for line in md.splitlines():
        if line.startswith("# "):
            continue
        if line.startswith("| "):
            in_table = True; table_lines.append(line); continue
        if in_table:
            rows = [[c.strip() for c in row.strip("|").split("|")] for row in table_lines if "---" not in row]
            if rows:
                t = Table(rows, repeatRows=1, hAlign="LEFT")
                t.setStyle(TableStyle([("BACKGROUND",(0,0),(-1,0),colors.HexColor("#DDEAF2")),("GRID",(0,0),(-
    1,-1),.3,colors.grey),("FONTSIZE",(0,0),(-1,-1),6.8),("VALIGN",(0,0),(-1,-1),"TOP"),("PADDING",(0,0),(-1,-
    1),3)])); story += [t, Spacer(1, 8)]
            in_table=False; table_lines=[]
        if line.startswith("## "):
            story += [Paragraph(html.escape(line[3:]), styles["Heading1"]), Spacer(1, 4)]
        elif line.startswith("- "):
            story.append(Paragraph("• " + html.escape(line[2:]).replace("`", ""), styles["BodyText"]))
        elif line.strip():
            story.append(Paragraph(html.escape(line).replace("**", "").replace("`", ""), styles["BodyText"]))
            story.append(Spacer(1, 4))
    if in_table and table_lines:
        rows = [[c.strip() for c in row.strip("|").split("|")] for row in table_lines if "---" not in row]
        story.append(Table(rows, repeatRows=1))
```

```
    story += [PageBreak(), Paragraph("Figures and Actual Output Evidence", styles["Heading1"])]
    for image_path in sorted((ROOT / "reports/figures").glob("*.png")):
        story += [Paragraph(image_path.stem.replace("_", " ").title(), styles["Heading2"]),
    Image(str(image_path), width=16.5*cm, height=9.5*cm, kind="proportional"), PageBreak()]
    for image_path in sorted((ROOT / "reports/screenshots").glob("*.png")):
        story += [Paragraph(image_path.stem.replace("_", " ").title(), styles["Heading2"]),
    Image(str(image_path), width=16.5*cm, height=9.5*cm, kind="proportional"), PageBreak()]
    if include_code:
        story += [Paragraph("Complete Code Appendix", styles["Heading1"])]
        code_files = (sorted((ROOT / "src/mit8301_credit_risk").glob("*.py")) +
                      sorted((ROOT / "scripts").glob("*.py")) +
                      sorted((ROOT / "tests").glob("*.py")))
        for code_path in code_files:
            if not code_path.exists(): continue
            story += [Paragraph(str(code_path.relative_to(ROOT)).replace("\\", "/"), styles["Heading2"])]
            wrapped=[]
            for number, line in enumerate(code_path.read_text(encoding="utf-8").splitlines(), 1):
                chunks = textwrap.wrap(line, width=112, subsequent_indent="    ", replace_whitespace=False,
    drop_whitespace=False) or [""]
                wrapped.append(f"{number:04d} {chunks[0]}"); wrapped.extend("     " + c for c in chunks[1:])
            story += [Preformatted("\n".join(wrapped), styles["SmallCode"]), PageBreak()]
    doc.build(story, onFirstPage=page_number, onLaterPages=page_number)


def main():
    results = json.loads(RESULTS.read_text(encoding="utf-8"))
    build_notebook(results)
    md = report_markdown(results)
    (ROOT / "reports/MIT8301_CA_Report.md").write_text(md, encoding="utf-8")
    build_pdf(ROOT / "reports/MIT8301_CA_Report.pdf", results, include_code=True)
    build_pdf(ROOT / "submission/Basil_Oforbuike_Emeokoro_MIT8301_CA.pdf", results, include_code=True)
    print("Notebook, report, and final submission PDF generated.")


if __name__ == "__main__":
    main()
```

scripts/regenerate_evidence.py

```
import json
from run_pipeline import evidence_card

d = json.load(open("reports/tables/pipeline_results.json", encoding="utf-8"))
evidence_card("Dataset loading and validation", [
    "Rows: 1,000 | Predictors: 9 | Target: credit_risk",
    "Row-level predictor reconciliation: PASS (100%)",
    "Mismatched predictor cells: 0",
    "Class balance: 700 good / 300 bad",
    "Missing: Saving accounts=183; Checking account=394",
], "01_dataset_validation.png")
evidence_card("Model execution evidence", [
    *[f"{x['Model']}: AUC={x['ROC_AUC']:.3f}, Recall={x['Recall']:.3f}, F1={x['F1']:.3f}" for x in
    d["metrics"]],
    f"Selected model: {d['selected_model']}",
    f"Scratch convergence: {d['scratch']['converged']}; iterations={d['scratch']['iterations']}",
], "02_model_results.png")
evidence_card("Pipeline completion", [
    f"Execution UTC: {d['environment']['executed_utc']}",
    f"Python {d['environment']['python']} | scikit-learn {d['environment']['scikit_learn']}",
    "Target validation: PASS", "EDA figures: 10 | Evaluation figures: 5",
    "Training rows: 800 | Untouched test rows: 200",
], "03_pipeline_completion.png")
print("Evidence cards regenerated with opaque white backgrounds.")
```

scripts/run_pipeline.py

```
"""Execute target validation, EDA, modelling, evaluation, and evidence generation."""
from __future__ import annotations

import json
import os
import platform
import sys
import warnings
from datetime import datetime, timezone
from pathlib import Path

ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(ROOT / "src"))

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
import sklearn
from sklearn.base import clone
from sklearn.metrics import (accuracy_score, classification_report, confusion_matrix,
                             ConfusionMatrixDisplay, f1_score, precision_score,
                             recall_score, roc_auc_score, RocCurveDisplay)
from sklearn.model_selection import GridSearchCV, StratifiedKFold, train_test_split
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.naive_bayes import GaussianNB
from sklearn.svm import SVC

from mit8301_credit_risk.config import RANDOM_STATE
from mit8301_credit_risk.data_validation import validate_and_restore
from mit8301_credit_risk.logistic_regression_scratch import LogisticRegressionScratch
from mit8301_credit_risk.preprocessing import build_preprocessor

FIG = ROOT / "reports" / "figures"
TABLE = ROOT / "reports" / "tables"
SHOT = ROOT / "reports" / "screenshots"
for directory in (FIG, TABLE, SHOT):
    directory.mkdir(parents=True, exist_ok=True)

sns.set_theme(style="whitegrid", context="notebook")
warnings.filterwarnings("ignore", category=FutureWarning, module="sklearn")
warnings.filterwarnings("ignore", message="Inconsistent values: penalty=")


def save(name: str):
    plt.tight_layout()
    plt.savefig(FIG / name, dpi=220, bbox_inches="tight")
    plt.close()


def eda_figures(df: pd.DataFrame) -> dict:
    y = df["credit_risk"]
    X = df.drop(columns="credit_risk")
    counts = y.map({0: "Good", 1: "Bad / risky"}).value_counts()
    counts.plot.bar(color=["#35618D", "#B64B4B"])
    plt.title("Credit-risk class distribution"); plt.xlabel("Class"); plt.ylabel("Applicants")
    save("01_target_distribution.png")
    for number, column in enumerate(["Age", "Credit amount", "Duration"], start=2):
        plt.hist(X[column], bins=25, color="#35618D", edgecolor="white")
        plt.title(f"Distribution of {column}"); plt.xlabel(column); plt.ylabel("Frequency")
        save(f"{number:02d}_{column.lower().replace(' ', '_')}_histogram.png")
    melted = X[["Age", "Credit amount", "Duration"]].apply(lambda s:
    (s-s.median())/(s.quantile(.75)-s.quantile(.25))).melt()
    sns.boxplot(data=melted, x="variable", y="value", color="#8CA9C2")
    plt.title("Numerical boxplots (robustly scaled)"); plt.xlabel("Feature"); plt.ylabel("Robust-scaled value")
    save("05_numerical_boxplots.png")
    missing = X.isna().sum().sort_values(ascending=False)
    missing[missing > 0].plot.bar(color="#B8863B")
    plt.title("Missing values by feature"); plt.xlabel("Feature"); plt.ylabel("Missing rows")
    save("06_missing_values.png")
    categories = ["Sex", "Job", "Housing", "Purpose"]
    fig, axes = plt.subplots(2, 2, figsize=(13, 9))
    for column, ax in zip(categories, axes.flat):
        X[column].astype(str).value_counts().plot.bar(ax=ax, color="#557A95")
        ax.set_title(column); ax.set_xlabel(""); ax.set_ylabel("Applicants"); ax.tick_params(axis="x",
    rotation=35)
    save("07_categorical_distributions.png")
    corr = df[["Age", "Credit amount", "Duration", "credit_risk"]].corr()
    sns.heatmap(corr, annot=True, fmt=".2f", cmap="vlag", center=0)
    plt.title("Numerical correlation matrix")
    save("08_correlation_heatmap.png")
    fig, axes = plt.subplots(1, 3, figsize=(14, 4))
    for column, ax in zip(["Age", "Credit amount", "Duration"], axes):
        sns.boxplot(data=df, x="credit_risk", y=column, ax=ax, color="#8CA9C2")
        ax.set_xticks([0, 1], ["Good", "Bad / risky"]); ax.set_xlabel("Credit class")
    save("09_numerical_features_by_target.png")
    risk_by_purpose = df.groupby("Purpose", dropna=False)["credit_risk"].mean().sort_values()
    (risk_by_purpose * 100).plot.barh(color="#7D5A6A")
    plt.title("Observed bad-credit rate by purpose"); plt.xlabel("Bad-credit rate (%)"); plt.ylabel("Purpose")
    save("10_risk_rate_by_purpose.png")
    numeric = X[["Age", "Credit amount", "Duration"]]
    q1, q3 = numeric.quantile(.25), numeric.quantile(.75)
    iqr = q3-q1
    outliers = ((numeric < q1-1.5*iqr) | (numeric > q3+1.5*iqr)).sum().to_dict()
    return outliers


```

```
def metric_row(name, y_true, prediction, probability):
    return {
        "Model": name,
        "Accuracy": accuracy_score(y_true, prediction),
        "Precision": precision_score(y_true, prediction, zero_division=0),
        "Recall": recall_score(y_true, prediction, zero_division=0),
        "F1": f1_score(y_true, prediction, zero_division=0),
        "ROC_AUC": roc_auc_score(y_true, probability),
    }


def evidence_card(title: str, lines: list[str], filename: str):
    fig = plt.figure(figsize=(12, 7), facecolor="white")
    plt.axis("off")
    plt.text(.04, .94, title, fontsize=20, weight="bold", va="top", color="#183B56")
    plt.text(.04, .84, "\n".join(lines), fontsize=12, family="monospace", va="top", linespacing=1.55)
    plt.savefig(SHOT / filename, dpi=180, bbox_inches="tight", facecolor="white", transparent=False)
    plt.close(fig)


def main():
    validation = validate_and_restore(
        ROOT / "data/raw/german_credit_data.csv",
        ROOT / "data/raw/target_source/german_credit_with_risk.csv",
        ROOT / "data/processed/german_credit_with_verified_target.csv",
        ROOT / "reports/data_provenance_and_target_validation.md",
    )
    df = pd.read_csv(ROOT / "data/processed/german_credit_with_verified_target.csv")
    outliers = eda_figures(df)
    X, y = df.drop(columns="credit_risk"), df["credit_risk"]
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=.20, random_state=RANDOM_STATE, stratify=y
    )
    cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)
    searches = {
        "Tuned Logistic Regression": (
            LogisticRegression(max_iter=3000, random_state=RANDOM_STATE),
            {"model__C": [.01, .1, 1, 10, 100], "model__penalty": ["l1", "l2"], "model__solver": ["liblinear"],
    "model__class_weight": [None, "balanced"]},
        ),
        "Tuned SVM": (
            SVC(probability=True, random_state=RANDOM_STATE),
            {"model__C": [.1, 1, 10], "model__kernel": ["linear", "rbf"], "model__gamma": ["scale", "auto", .01,
     .1], "model__class_weight": [None, "balanced"]},
        ),
        "Tuned Gaussian Naive Bayes": (
            GaussianNB(), {"model__var_smoothing": np.logspace(-12, -6, 7)}
        ),
    }
    metrics, fitted, tuning = [], {}, {}
    for name, (model, grid) in searches.items():
        pipeline = Pipeline([("preprocess", build_preprocessor(X)), ("model", model)])
        search = GridSearchCV(pipeline, grid, cv=cv, scoring="roc_auc", n_jobs=-1, return_train_score=False)
        search.fit(X_train, y_train)
        prediction = search.predict(X_test)
        probability = search.predict_proba(X_test)[:, 1]
        metrics.append(metric_row(name, y_test, prediction, probability))
        fitted[name] = search.best_estimator_
        tuning[name] = {"best_params": search.best_params_, "best_cv_roc_auc": float(search.best_score_)}

    scratch_preprocessor = build_preprocessor(X)
    X_train_t = scratch_preprocessor.fit_transform(X_train, y_train)
    X_test_t = scratch_preprocessor.transform(X_test)
    scratch = LogisticRegressionScratch().fit(X_train_t, y_train.to_numpy())
    scratch_pred = scratch.predict(X_test_t)
    scratch_prob = scratch.predict_proba(X_test_t)[:, 1]
    metrics.append(metric_row("Logistic Regression from Scratch", y_test, scratch_pred, scratch_prob))
    plt.plot(scratch.loss_history_, color="#35618D")
    plt.title("Scratch logistic-regression convergence"); plt.xlabel("Iteration"); plt.ylabel("Binary cross-
    entropy + L2")
    save("11_logistic_regression_scratch_convergence.png")

    comparison = pd.DataFrame(metrics).sort_values("ROC_AUC", ascending=False).reset_index(drop=True)
    comparison.to_csv(TABLE / "model_comparison.csv", index=False)
    header = "| Model | Accuracy | Precision | Recall | F1 | ROC-AUC |\n|---|---:|---:|---:|---:|---:|\n"
    rows = "\n".join(
        f"| {r.Model} | {r.Accuracy:.3f} | {r.Precision:.3f} | {r.Recall:.3f} | {r.F1:.3f} | {r.ROC_AUC:.3f} |"
        for r in comparison.itertuples()
    )
    (TABLE / "model_comparison.md").write_text(header + rows + "\n", encoding="utf-8")
    pd.DataFrame([
        {"Model": k, "Best CV ROC-AUC": v["best_cv_roc_auc"], "Best parameters": json.dumps(v["best_params"])}
        for k, v in tuning.items()
    ]).to_csv(TABLE / "hyperparameter_tuning.csv", index=False)

    comparison.set_index("Model")[["Accuracy", "Precision", "Recall", "F1", "ROC_AUC"]].plot.bar(figsize=(12,
    6))
    plt.title("Test-set model comparison"); plt.ylabel("Score"); plt.ylim(0, 1); plt.legend(ncol=5, loc="lower
    center", bbox_to_anchor=(.5, -0.32))
    save("12_model_metric_comparison.png")
    fig, ax = plt.subplots(figsize=(8, 6))
    for name, model in fitted.items():
        RocCurveDisplay.from_estimator(model, X_test, y_test, name=name, ax=ax)
    RocCurveDisplay.from_predictions(y_test, scratch_prob, name="Scratch Logistic", ax=ax)
    ax.plot([0, 1], [0, 1], "--", color="grey"); ax.set_title("Test-set ROC curves")
    save("13_roc_curves.png")
    fig, axes = plt.subplots(2, 2, figsize=(11, 9))
    all_predictions = {name: model.predict(X_test) for name, model in fitted.items()}
    all_predictions["Scratch Logistic"] = scratch_pred
    for ax, (name, prediction) in zip(axes.flat, all_predictions.items()):
        ConfusionMatrixDisplay(confusion_matrix(y_test, prediction), display_labels=["Good", "Bad"]).plot(ax=ax,
     colorbar=False)
        ax.set_title(name)
    save("14_confusion_matrices.png")
```

```
    logistic = fitted["Tuned Logistic Regression"]
    names = logistic.named_steps["preprocess"].get_feature_names_out()
    coefficients = logistic.named_steps["model"].coef_[0]
    coef = pd.DataFrame({"Feature": names, "Coefficient": coefficients})
    coef["Absolute coefficient"] = coef["Coefficient"].abs()
    coef = coef.sort_values("Absolute coefficient", ascending=False)
    coef.to_csv(TABLE / "logistic_feature_coefficients.csv", index=False)
    top = coef.head(15).sort_values("Coefficient")
    top.plot.barh(x="Feature", y="Coefficient", legend=False, color=["#35618D" if x < 0 else "#B64B4B" for x in top["Coefficient"]])
    plt.title("Most influential logistic-regression coefficients"); plt.xlabel("Coefficient (positive = higher bad-credit log-odds)"); plt.ylabel("")
    save("15_feature_importance.png")

    best_name = comparison.iloc[0]["Model"]
    selected_name = "Tuned Logistic Regression" if comparison.loc[comparison.Model == "Tuned Logistic Regression", "ROC_AUC"].iloc[0] >= comparison.ROC_AUC.max() - .03 else best_name
    selected = fitted.get(selected_name, fitted[best_name])
    fairness = []
    selected_pred = selected.predict(X_test)
    for group in sorted(X_test["Sex"].unique()):
        mask = X_test["Sex"].eq(group).to_numpy()
        fairness.append({"Sex": group, "n": int(mask.sum()), "Accuracy": accuracy_score(y_test.to_numpy()[mask], selected_pred[mask]), "Recall_bad": recall_score(y_test.to_numpy()[mask], selected_pred[mask], zero_division=0)})
    pd.DataFrame(fairness).to_csv(TABLE / "fairness_by_sex.csv", index=False)
    reports = {name: classification_report(y_test, pred, output_dict=True) for name, pred in all_predictions.items()}
    results = {
        "validation": validation, "shape": list(df.shape), "missing_counts": X.isna().sum().to_dict(),
        "outlier_counts_iqr": outliers, "train_rows": len(X_train), "test_rows": len(X_test),
        "tuning": tuning, "metrics": comparison.to_dict(orient="records"), "selected_model": selected_name,
        "selection_rule": "Prefer the tuned interpretable logistic model when within 0.03 ROC-AUC of the highest model; otherwise select the highest ROC-AUC model.",
        "scratch": {"iterations": scratch.n_iter_, "converged": scratch.converged_, "final_loss": scratch.loss_history_[-1]},
        "top_coefficients": coef.head(10).to_dict(orient="records"), "fairness_by_sex": fairness,
        "classification_reports": reports,
        "environment": {"python": sys.version.split()[0], "platform": platform.platform(), "pandas": pd.__version__, "numpy": np.__version__, "scikit_learn": sklearn.__version__, "executed_utc": datetime.now(timezone.utc).isoformat()},
    }
    (TABLE / "pipeline_results.json").write_text(json.dumps(results, indent=2), encoding="utf-8")
    evidence_card("Dataset loading and validation", [
        "Rows: 1,000 | Predictors: 9 | Target: credit_risk",
        "Row-level predictor reconciliation: PASS (100%)",
        "Mismatched predictor cells: 0",
        "Class balance: 700 good / 300 bad",
        "Missing: Saving accounts=183; Checking account=394",
    ], "01_dataset_validation.png")
    evidence_card("Model execution evidence", [
        *[f"{r.Model}: AUC={r.ROC_AUC:.3f}, Recall={r.Recall:.3f}, F1={r.F1:.3f}" for r in comparison.itertuples()],
        f"Selected model: {selected_name}",
        f"Scratch convergence: {scratch.converged_}; iterations={scratch.n_iter_}",
    ], "02_model_results.png")
    evidence_card("Pipeline completion", [
        f"Execution UTC: {results['environment']['executed_utc']}",
        f"Python {results['environment']['python']} | scikit-learn {sklearn.__version__}",
        "Target validation: PASS",
        "EDA figures: 10 | Evaluation figures: 5",
        "Training rows: 800 | Untouched test rows: 200",
    ], "03_pipeline_completion.png")
    print(json.dumps({"selected_model": selected_name, "metrics": results["metrics"], "validation": "PASS"}, indent=2))


if __name__ == "__main__":
    main()
```

scripts/validate_submission.py

```
"""Fail-fast validation for the single-attempt LMS submission."""
from __future__ import annotations

import json
from pathlib import Path

import pdfplumber

ROOT = Path(__file__).resolve().parents[1]
CORRECT_EMAIL = "b.emeokoro6653@miva.edu.ng"
DISALLOWED_EMAIL = CORRECT_EMAIL.rsplit(".", 1)[0] + ".org"


def main():
    required = [
        "README.md", "notebooks/MIT8301_CA_German_Credit_Risk.ipynb",
        "reports/MIT8301_CA_Report.md", "reports/MIT8301_CA_Report.pdf",
        "reports/data_provenance_and_target_validation.md", "reports/tables/model_comparison.csv",
        "submission/Basil_Oforbuike_Emeokoro_MIT8301_CA.pdf",
    ]
    missing = [name for name in required if not (ROOT / name).exists()]
    assert not missing, f"Missing required files: {missing}"
    results = json.loads((ROOT / "reports/tables/pipeline_results.json").read_text(encoding="utf-8"))
    assert results["validation"]["validation_passed"] and results["validation"]["total_mismatched_cells"] == 0
    assert len(results["metrics"]) == 4 and len(list((ROOT / "reports/figures").glob("*.png"))) >= 15
    notebook = json.loads((ROOT / "notebooks/MIT8301_CA_German_Credit_Risk.ipynb").read_text(encoding="utf-8"))
    assert all(cell.get("execution_count") is not None for cell in notebook["cells"] if cell["cell_type"] ==
    "code")
    text_files = list(ROOT.rglob("*.md")) + list(ROOT.rglob("*.py")) + list(ROOT.rglob("*.ipynb"))
    combined = "\n".join(path.read_text(encoding="utf-8") for path in text_files)
    assert DISALLOWED_EMAIL not in combined
    assert CORRECT_EMAIL in combined
    final_pdf = ROOT / "submission/Basil_Oforbuike_Emeokoro_MIT8301_CA.pdf"
    with pdfplumber.open(final_pdf) as pdf:
        page_count = len(pdf.pages)
        assert page_count >= 20
        first_text = "\n".join((page.extract_text() or "") for page in pdf.pages[:3])
        assert CORRECT_EMAIL in first_text
    checklist = f"""# Final Submission Checklist

- [x] Target-source reconciliation passed with zero mismatched cells.
- [x] Verified 1,000 rows and 700/300 class balance.
- [x] Notebook contains executed outputs.
- [x] Accuracy, precision, recall, F1, and ROC-AUC are complete for four models.
- [x] At least 15 actual-output figures exist.
- [x] Report and final single-file PDF exist and are non-empty.
- [x] Student details use {CORRECT_EMAIL} consistently.
- [x] All 40 rubric marks are mapped in the report.
- [x] Unit and smoke tests pass.
- [x] No credentials or machine-specific paths appear in user instructions.
"""
    (ROOT / "submission/submission_checklist.md").write_text(checklist, encoding="utf-8")
    print(f"PASS: {len(required)} required artifacts, {len(list((ROOT / 'reports/figures').glob('*.png')))}
    figures, {page_count} PDF pages")


if __name__ == "__main__":
    main()
```

scripts/validate_target_source.py

```
0001 from run_pipeline import main
0002
0003 if __name__ == "__main__":
0004     main()
```

tests/test_data_validation.py

```
0001 from pathlib import Path
0002 import pandas as pd
0003
0004 from mit8301_credit_risk.data_validation import validate_and_restore
0005
0006
0007 ROOT = Path(__file__).resolve().parents[1]
0008
0009
0010 def test_exact_target_reconciliation(tmp_path):
0011     output = tmp_path / "restored.csv"
0012     report = tmp_path / "report.md"
0013     result = validate_and_restore(ROOT / "data/raw/german_credit_data.csv", ROOT /
         "data/raw/target_source/german_credit_with_risk.csv", output, report)
0014     restored = pd.read_csv(output)
0015     assert result["validation_passed"]
0016     assert result["total_mismatched_cells"] == 0
0017     assert restored.shape == (1000, 10)
0018     assert restored.credit_risk.value_counts().to_dict() == {0: 700, 1: 300}
0019
0020
0021 def test_mismatch_stops_reconciliation(tmp_path):
0022     supplied = pd.DataFrame({"Age": [20], "Sex": ["male"]})
0023     candidate = pd.DataFrame({"Age": [21], "Sex": ["male"], "Risk": ["good"]})
0024     supplied.to_csv(tmp_path / "a.csv", index=False); candidate.to_csv(tmp_path / "b.csv", index=False)
0025     try:
0026         validate_and_restore(tmp_path / "a.csv", tmp_path / "b.csv", tmp_path / "o.csv", tmp_path / "r.md")
0027     except ValueError as exc:
0028         assert "mismatched" in str(exc)
0029     else:
0030         raise AssertionError("A predictor mismatch must stop target restoration")
```

tests/test_logistic_regression_scratch.py

```
0001 import numpy as np
0002 from sklearn.datasets import make_classification
0003 from sklearn.linear_model import LogisticRegression
0004 from sklearn.metrics import accuracy_score
0005 from sklearn.preprocessing import StandardScaler
0006
0007 from mit8301_credit_risk.logistic_regression_scratch import LogisticRegressionScratch
0008
0009
0010 def test_sigmoid_and_probability_bounds():
0011     values = LogisticRegressionScratch._sigmoid(np.array([-1000, 0, 1000]))
0012     assert np.all((values >= 0) & (values <= 1))
0013     assert values[1] == 0.5
0014
0015
0016 def test_loss_decreases_and_predictions_agree_reasonably():
0017     X, y = make_classification(n_samples=500, n_features=8, random_state=42)
0018     X = StandardScaler().fit_transform(X)
0019     model = LogisticRegressionScratch(max_iter=3000).fit(X, y)
0020     benchmark = LogisticRegression(max_iter=2000).fit(X, y)
0021     assert model.loss_history_[-1] < model.loss_history_[0]
0022     assert set(model.predict(X)) <= {0, 1}
0023     assert np.all((model.predict_proba(X) >= 0) & (model.predict_proba(X) <= 1))
0024     assert abs(accuracy_score(y, model.predict(X)) - accuracy_score(y, benchmark.predict(X))) < 0.08
```

tests/test_pipeline_smoke.py

```
0001 import json
0002 from pathlib import Path
0003
0004 ROOT = Path(__file__).resolve().parents[1]
0005
0006
0007 def test_generated_pipeline_outputs_are_complete():
0008     results = json.loads((ROOT / "reports/tables/pipeline_results.json").read_text(encoding="utf-8"))
0009     assert results["validation"]["validation_passed"]
0010     assert len(results["metrics"]) == 4
0011     for model in results["metrics"]:
0012         assert {"Accuracy", "Precision", "Recall", "F1", "ROC_AUC"} <= model.keys()
0013     assert (ROOT / "submission/Basil_Oforbuike_Emeokoro_MIT8301_CA.pdf").stat().st_size > 100_000
```

tests/test_preprocessing.py

```
from pathlib import Path
import pandas as pd
from sklearn.model_selection import train_test_split

from mit8301_credit_risk.preprocessing import build_preprocessor


ROOT = Path(__file__).resolve().parents[1]


def test_preprocessor_fits_training_data_and_handles_unknowns():
    df = pd.read_csv(ROOT / "data/processed/german_credit_with_verified_target.csv")
    X_train, X_test = train_test_split(df.drop(columns="credit_risk"), test_size=.2, random_state=42)
    processor = build_preprocessor(X_train).fit(X_train)
    transformed_train = processor.transform(X_train)
    changed = X_test.copy(); changed.loc[changed.index[0], "Purpose"] = "unseen-purpose"
    transformed_test = processor.transform(changed)
    assert transformed_train.shape[1] == transformed_test.shape[1]
    assert hasattr(processor.named_transformers_["num"].named_steps["clipper"], "lower_")
```